

Contents

GW2 Trading Post Analyst	15
Specification, architecture, roadmap, milestones, tickets and Qwen/MTPLX development prompts	15
How to use this package	16
Purpose	16
Development environment	16
Important editor note	16
Deliverables in this package	16
How to use this package	17
Normative language	17
Project Specification - GW2 Trading Post Analyst	17
1. Product statement	17
2. Product principles	18
3. Scope	18
MVP scope	18
Explicitly out of MVP scope	19
4. Users	19
5. Main user journeys	19
Journey A - Market flip scan	19
Journey B - Account-aware crafting	19
Journey C - Session plan	20
6. Definitions	20
Profit	20
Realized profit	20
Unrealized P/L	20
Liquidity	20
Confidence	20
7. Non-functional requirements	20
8. API verification policy	21
9. Market analysis requirements	21
Top-of-book	21
Order-book analysis	21
Fee configuration	21
Data freshness	22
10. Crafting requirements	22
11. Session planning requirements	22
12. History requirements	22
13. Future historical investment feature	23
14. Future LLM boundary	23
15. Completion standard	23
Architecture	23

Target stack	23
Runtime topology	24
Layers	24
Domain	24
Application	24
Infrastructure	24
Analytics	24
Web	24
Recommended repository structure	24
External API gateway	25
API caching	25
Rate-limit strategy	25
Persistence	25
Money	26
Secrets	26
Local network binding	26
Error taxonomy	26
Observability	26
LLM separation	27
Security and Data Protection Baseline	27
Scope	27
Mandatory/conditional requirements	27
Best practices required by this project	27
API token specifics	28
Data minimization	28
Threat model	28
Read-only enforcement	28
Testing Strategy	29
Quality layers	29
Unit tests	29
Integration tests	29
Contract/smoke tests	29
Browser tests	29
Test fixture policy	30
Specification-test coupling	30
Financial correctness gates	30
LLM testing	30
UI/UX Specification	31
Visual direction	31
Main screens	31
Dashboard	31
Opportunity detail	31
Crafting detail	31

Account/settings	32
Interaction rules	32
Desktop-first	32
Qwen + MTPLX Development Workflow	32
Role of Qwen	32
Context hierarchy	32
Ticket loop	32
When Qwen should stop	33
Prompt style	33
Commit guidance	33
ADRs	33
Review rules	33
Roadmap	34
External Verification Notes	34
MTPLX	34
Qwen3.8-27B GGUF	35
Visual Studio on Mac	35
GW2 API	35
Architecture Decision Records	35
ADR-001 - Application Stack	35
Status	35
Decision	35
Rationale	35
Alternative	36
ADR-002 - Local-First Deployment	36
Status	36
Decision	36
Consequences	36
ADR-003 - No Application LLM in Current Scope	36
Status	36
Decision	36
Rationale	36
Future	36
ADR-004 - Single GW2 API Gateway	37
Status	37
Decision	37
Rationale	37
ADR-005 - Integer Copper Money	37

Status	37
Decision	37
Rationale	37
ADR-006 - Local Secret Storage	37
Status	37
Decision	37
Development fallback	37
ADR-007 - Read-Only External Boundary	38
Status	38
Decision	38
Rationale	38
Milestones	38
M0 - Discovery and external-contract validation	38
Goal	38
Dependencies	38
Completion rule	38
M1 - Repository and development foundation	38
Goal	38
Dependencies	38
Completion rule	39
M2 - GW2 data gateway and caching	39
Goal	39
Dependencies	39
Completion rule	39
M3 - Deterministic market engine	39
Goal	39
Dependencies	39
Completion rule	39
M4 - Dashboard and session planning	39
Goal	39
Dependencies	40
Completion rule	40
M5 - Account-aware analysis and crafting	40
Goal	40
Dependencies	40
Completion rule	40
M6 - Personal history and reconciliation	40

Goal	40
Dependencies	40
Completion rule	40
M7 - Historical market data and investment research	41
Goal	41
Dependencies	41
Completion rule	41
M8 - Hardening, accessibility, release readiness	41
Goal	41
Dependencies	41
Completion rule	41
AI Context Files	41
Permanent Context for Qwen	41
Identity	41
Hard constraints	42
Target stack	42
Local development	42
Current architecture	42
Model/runtime note	42
Required behavior	42
Context - M0	43
Milestone	43
Goal	43
Permanent context	43
Rules for this milestone	43
Context - M1	43
Milestone	43
Goal	43
Permanent context	43
Rules for this milestone	43
Context - M2	43
Milestone	43
Goal	44
Permanent context	44
Rules for this milestone	44
Context - M3	44
Milestone	44
Goal	44
Permanent context	44

Rules for this milestone	44
Context - M4	44
Milestone	44
Goal	44
Permanent context	45
Rules for this milestone	45
Context - M5	45
Milestone	45
Goal	45
Permanent context	45
Rules for this milestone	45
Context - M6	45
Milestone	45
Goal	45
Permanent context	45
Rules for this milestone	46
Context - M7	46
Milestone	46
Goal	46
Permanent context	46
Rules for this milestone	46
Context - M8	46
Milestone	46
Goal	46
Permanent context	46
Rules for this milestone	46
Implementation Tickets and Qwen Prompts	47
TKT-M0-01 - Verify MTPLX/Qwen3.8-27B development model	
path	47
Milestone	47
Goal	47
Dependencies	47
Acceptance criteria	47
Required tests	47
Non-goals	47
Specification references	47
Agent prompt	48
Dedicated execution prompt	48
TKT-M0-02 - Validate GW2 API endpoint and permission matrix	49

Milestone	49
Goal	49
Dependencies	49
Acceptance criteria	49
Required tests	49
Non-goals	49
Specification references	49
Agent prompt	49
Dedicated execution prompt	50
TKT-M0-03 - Validate API quotas, caching guidance, and error behavior	50
Milestone	50
Goal	51
Dependencies	51
Acceptance criteria	51
Required tests	51
Non-goals	51
Specification references	51
Agent prompt	51
Dedicated execution prompt	51
TKT-M0-04 - Validate legal/security scope and read-only boundary	52
Milestone	52
Goal	52
Dependencies	52
Acceptance criteria	53
Required tests	53
Non-goals	53
Specification references	53
Agent prompt	53
Dedicated execution prompt	53
TKT-M1-01 - Initialize repository and solution skeleton	54
Milestone	54
Goal	54
Dependencies	54
Acceptance criteria	54
Required tests	55
Non-goals	55
Specification references	55
Agent prompt	55
Dedicated execution prompt	55
TKT-M1-02 - Add local configuration and secret-store abstraction	56

Milestone	56
Goal	56
Dependencies	56
Acceptance criteria	56
Required tests	56
Non-goals	56
Specification references	57
Agent prompt	57
Dedicated execution prompt	57
TKT-M1-03 - Add test infrastructure and fixture conventions	58
Milestone	58
Goal	58
Dependencies	58
Acceptance criteria	58
Required tests	58
Non-goals	58
Specification references	58
Agent prompt	58
Dedicated execution prompt	58
TKT-M1-04 - Add local-only runtime and baseline security mid-	
dleware	59
Milestone	59
Goal	59
Dependencies	60
Acceptance criteria	60
Required tests	60
Non-goals	60
Specification references	60
Agent prompt	60
Dedicated execution prompt	60
TKT-M2-01 - Implement typed GW2 API client for public market	
data	61
Milestone	61
Goal	61
Dependencies	61
Acceptance criteria	61
Required tests	62
Non-goals	62
Specification references	62
Agent prompt	62
Dedicated execution prompt	62
TKT-M2-02 - Build request scheduler, deduplication, and retry	

policy	63
Milestone	63
Goal	63
Dependencies	63
Acceptance criteria	63
Required tests	63
Non-goals	63
Specification references	64
Agent prompt	64
Dedicated execution prompt	64
TKT-M2-03 - Implement cache with freshness metadata	65
Milestone	65
Goal	65
Dependencies	65
Acceptance criteria	65
Required tests	65
Non-goals	65
Specification references	65
Agent prompt	65
Dedicated execution prompt	65
TKT-M2-04 - Add market-data observability and diagnostics	66
Milestone	66
Goal	66
Dependencies	66
Acceptance criteria	67
Required tests	67
Non-goals	67
Specification references	67
Agent prompt	67
Dedicated execution prompt	67
TKT-M3-01 - Implement Money and fee policy	68
Milestone	68
Goal	68
Dependencies	68
Acceptance criteria	68
Required tests	68
Non-goals	68
Specification references	69
Agent prompt	69
Dedicated execution prompt	69
TKT-M3-02 - Implement order-book quantity simulation	70
Milestone	70

Goal	70
Dependencies	70
Acceptance criteria	70
Required tests	70
Non-goals	70
Specification references	70
Agent prompt	70
Dedicated execution prompt	71
TKT-M3-03 - Implement flip profitability and liquidity analysis	71
Milestone	71
Goal	72
Dependencies	72
Acceptance criteria	72
Required tests	72
Non-goals	72
Specification references	72
Agent prompt	72
Dedicated execution prompt	72
TKT-M3-04 - Implement deterministic opportunity scoring	73
Milestone	73
Goal	73
Dependencies	73
Acceptance criteria	74
Required tests	74
Non-goals	74
Specification references	74
Agent prompt	74
Dedicated execution prompt	74
TKT-M4-01 - Create market dashboard and filters	75
Milestone	75
Goal	75
Dependencies	75
Acceptance criteria	75
Required tests	75
Non-goals	76
Specification references	76
Agent prompt	76
Dedicated execution prompt	76
TKT-M4-02 - Create opportunity detail view	77
Milestone	77
Goal	77
Dependencies	77

Acceptance criteria	77
Required tests	77
Non-goals	77
Specification references	77
Agent prompt	77
Dedicated execution prompt	78
TKT-M4-03 - Implement user session preferences	78
Milestone	78
Goal	78
Dependencies	79
Acceptance criteria	79
Required tests	79
Non-goals	79
Specification references	79
Agent prompt	79
Dedicated execution prompt	79
TKT-M4-04 - Implement session planner with coarse effort categories	80
Milestone	80
Goal	80
Dependencies	80
Acceptance criteria	80
Required tests	81
Non-goals	81
Specification references	81
Agent prompt	81
Dedicated execution prompt	81
TKT-M5-01 - Implement token validation and permission-aware account access	82
Milestone	82
Goal	82
Dependencies	82
Acceptance criteria	82
Required tests	82
Non-goals	82
Specification references	83
Agent prompt	83
Dedicated execution prompt	83
TKT-M5-02 - Implement account snapshot ingestion	84
Milestone	84
Goal	84
Dependencies	84

Acceptance criteria	84
Required tests	84
Non-goals	84
Specification references	84
Agent prompt	84
Dedicated execution prompt	84
TKT-M5-03 - Implement owned-item opportunity cost	85
Milestone	85
Goal	85
Dependencies	86
Acceptance criteria	86
Required tests	86
Non-goals	86
Specification references	86
Agent prompt	86
Dedicated execution prompt	86
TKT-M5-04 - Implement bounded crafting path search	87
Milestone	87
Goal	87
Dependencies	87
Acceptance criteria	87
Required tests	88
Non-goals	88
Specification references	88
Agent prompt	88
Dedicated execution prompt	88
TKT-M6-01 - Create operation history model and local persistence	89
Milestone	89
Goal	89
Dependencies	89
Acceptance criteria	89
Required tests	90
Non-goals	90
Specification references	90
Agent prompt	90
Dedicated execution prompt	90
TKT-M6-02 - Implement realized profit reconciliation	91
Milestone	91
Goal	91
Dependencies	91
Acceptance criteria	91
Required tests	91

Non-goals	91
Specification references	92
Agent prompt	92
Dedicated execution prompt	92
TKT-M6-03 - Build personal dashboard/history statistics	93
Milestone	93
Goal	93
Dependencies	93
Acceptance criteria	93
Required tests	93
Non-goals	93
Specification references	93
Agent prompt	93
Dedicated execution prompt	93
TKT-M6-04 - Implement local data clearing and backup guidance	94
Milestone	94
Goal	94
Dependencies	95
Acceptance criteria	95
Required tests	95
Non-goals	95
Specification references	95
Agent prompt	95
Dedicated execution prompt	95
TKT-M7-01 - Design local market snapshot schema and sampling policy	96
Milestone	96
Goal	96
Dependencies	96
Acceptance criteria	96
Required tests	97
Non-goals	97
Specification references	97
Agent prompt	97
Dedicated execution prompt	97
TKT-M7-02 - Implement economical historical collection scheduler	98
Milestone	98
Goal	98
Dependencies	98
Acceptance criteria	98
Required tests	98

Non-goals	98
Specification references	98
Agent prompt	99
Dedicated execution prompt	99
TKT-M7-03 - Implement descriptive historical analytics	100
Milestone	100
Goal	100
Dependencies	100
Acceptance criteria	100
Required tests	100
Non-goals	100
Specification references	100
Agent prompt	100
Dedicated execution prompt	101
TKT-M7-04 - Create investment-research view with uncertainty	101
Milestone	101
Goal	102
Dependencies	102
Acceptance criteria	102
Required tests	102
Non-goals	102
Specification references	102
Agent prompt	102
Dedicated execution prompt	102
TKT-M8-01 - Perform security review and secret-leak audit	103
Milestone	103
Goal	103
Dependencies	103
Acceptance criteria	103
Required tests	104
Non-goals	104
Specification references	104
Agent prompt	104
Dedicated execution prompt	104
TKT-M8-02 - Perform browser/accessibility and UX hardening	105
Milestone	105
Goal	105
Dependencies	105
Acceptance criteria	105
Required tests	105
Non-goals	105
Specification references	106

Agent prompt	106
Dedicated execution prompt	106
TKT-M8-03 - Performance and API-request efficiency review	107
Milestone	107
Goal	107
Dependencies	107
Acceptance criteria	107
Required tests	107
Non-goals	107
Specification references	107
Agent prompt	107
Dedicated execution prompt	107
TKT-M8-04 - Prepare local release package and handoff documenta-	108
tion	
Milestone	108
Goal	108
Dependencies	109
Acceptance criteria	109
Required tests	109
Non-goals	109
Specification references	109
Agent prompt	109
Dedicated execution prompt	109

GW2 Trading Post Analyst

Specification, architecture, roadmap, milestones, tickets and Qwen/MTPLX development prompts

Prepared for local read-only development on Apple Silicon. Verification date: 19 August 2026.

Critical runtime note: MTPLX is MLX-native. The raw `unsloth/Qwen3.8-27B-GGUF` repository is not assumed to load directly. Milestone M0 is an explicit compatibility gate; use an MTPLX-compatible Qwen3.8 27B artifact or an approved conversion path if required.

Application scope: The application itself contains no LLM integration. Qwen is the development agent only.

How to use this package

This repository is the complete planning and development package for a local, read-only Guild Wars 2 Trading Post analysis application.

Purpose

Build a browser-based local application that:

- reads Guild Wars 2 public market data and, optionally, account data through the GW2 API;
- minimizes API requests through batching, caching, deduplication, and scheduled refreshes;
- never automates gameplay, Trading Post actions, or API mutations;
- calculates market and crafting opportunities deterministically;
- makes assumptions, data age, uncertainty, liquidity, and opportunity cost visible;
- persists local user history without requiring a server or cloud account;
- can later support historical analysis and long-term investment research;
- may later add an application-facing LLM, but **no LLM integration is part of the MVP or current architecture.**

Development environment

The local coding agent is Qwen3.8-27B operated through MTPLX on an Apple Silicon Mac. MTPLX exposes an OpenAI-compatible local API and is used only as the development assistant/runtime. It is not an application runtime dependency.

Important: MTPLX currently runs MLX-native model artifacts and explicitly treats GGUF as the llama.cpp format. The raw `unsloth/Qwen3.8-27B-GGUF` repository must therefore be treated as a source model reference, not as a guaranteed MTPLX-loadable artifact. Milestone M0 contains an explicit compatibility gate before development proceeds.

Important editor note

Visual Studio for Mac was retired on August 31, 2024. On macOS use Visual Studio Code (or another current editor) with the local MTPLX-compatible coding workflow.

Deliverables in this package

- `docs/specs/project-spec.md` - normative product and system specification.
- `docs/architecture/architecture.md` - technical architecture.

- `docs/security/security.md` - security and French/EU data-protection baseline.
- `docs/testing/testing-strategy.md` - test strategy and quality gates.
- `docs/ux/ux.md` - UI/UX rules.
- `docs/workflow/ai-development-workflow.md` - development workflow for Qwen via MTPLX.
- `docs/adr/` - Architecture Decision Records (ADRs).
- `docs/milestones/` - milestone definitions and completion criteria.
- `docs/context/` - lightweight context files intended for Qwen.
- `docs/tickets/` - implementation tickets.
- `docs/prompts/` - one execution prompt per ticket.
- `config/` - repository configuration templates.

How to use this package

1. Create an empty Git repository.
2. Copy the contents of this package into it.
3. Read `docs/specs/project-spec.md` once yourself.
4. Load only `docs/context/permanent-context.md`, the current milestone context, and one ticket into Qwen at a time.
5. Execute tickets in numerical order within each milestone unless an explicit dependency says otherwise.
6. Require tests for every behavior change.
7. Do not let Qwen silently change architectural decisions. If a ticket exposes a real architectural issue, update or create an ADR first.
8. Keep the application fully read-only and local until a future scope explicitly changes that decision.

Normative language

- MUST = mandatory.
- MUST NOT = prohibited.
- SHOULD = strong recommendation; deviation requires rationale.
- MAY = optional.
- ASSUMPTION = provisional until verified.
- VERIFY = must be checked against current authoritative documentation or measured behavior before release.

Project Specification - GW2 Trading Post Analyst

1. Product statement

The system is a local, browser-based decision-support tool for Guild Wars 2 Trading Post and crafting analysis. Its purpose is to help a single user identify

potentially profitable in-game gold-making opportunities using legal, read-only analysis.

The application is not a trading bot, gameplay bot, order executor, browser automator, or autonomous game agent.

2. Product principles

1. **Read-only by design.** The application may request permitted read-only GW2 API resources. It **MUST NOT** automate Trading Post actions or gameplay.
2. **Deterministic financial truth.** Money calculations, fee calculations, opportunity ranking, and crafting feasibility are performed by deterministic application code. A future LLM **MAY** explain results but **MUST NOT** become the source of numerical truth.
3. **Data freshness is first-class.** Every market-derived result carries capture time and a data-quality/freshness state.
4. **API minimization.** All GW2 requests go through one server-side gateway with caching, batching, deduplication, scheduling, and retry/rate-limit handling.
5. **Explicit assumptions.** A result is a scenario, not a guarantee of execution or profit.
6. **Opportunity cost matters.** Owned items are never treated as free inputs by default.
7. **Local-first.** No server, account system, cloud database, or mandatory remote service is required for v1.
8. **Testability.** Core calculations are pure and independently testable.
9. **Secure by default.** The API key stays server-side/local and outside version control.
10. **Incremental complexity.** Features requiring historical data are deferred until local collection has created enough evidence.

3. Scope

MVP scope

- current market price and order-book ingestion;
- caching and request scheduling;
- fee-aware flipping scenarios;
- order-book depth simulation;
- opportunity ranking using deterministic scoring;
- user constraints: capital, strategy preference, minimum profit, risk tolerance, capital allocation, data freshness;
- local account connection with minimum required API permissions;
- account data snapshots for bank/materials and relevant character crafting information;
- limited-depth crafting profitability;

- owned-material opportunity-cost analysis;
- session planning using coarse effort categories;
- modern desktop browser UI;
- comprehensive unit/integration/fixture tests;
- local SQLite persistence;
- clear read-only disclosures and data freshness indicators.

Explicitly out of MVP scope

- any action that places/cancels TP orders;
- gameplay automation;
- browser automation against the game or TP;
- a public/multi-user server;
- cloud persistence;
- mandatory LLM integration;
- precise execution-time prediction;
- price forecasting marketed as reliable prediction;
- long-term investment recommendations based only on snapshots;
- unrestricted full-world crafting path optimization;
- automatic profit reconciliation with in-game state beyond available read-only transaction/account data;
- mobile-first layouts.

4. Users

Initial user: one human developer/player operating the tool locally on their own Mac.

No user-to-user sharing is required for v1.

5. Main user journeys

Journey A - Market flip scan

1. User opens the local dashboard.
2. Application shows when market data was last refreshed.
3. User chooses capital and risk/strategy preferences.
4. Application selects candidate items from available market data.
5. For each candidate the deterministic engine calculates scenario profit, ROI, capital need, price impact, liquidity proxy, and data freshness.
6. UI ranks and filters opportunities.
7. User opens an opportunity to see assumptions and calculation details.

Journey B - Account-aware crafting

1. User supplies an API key with the minimum necessary scopes.
2. Backend validates the token and reports available permissions without exposing the secret.

3. Account snapshots are fetched only when needed.
4. Crafting analyzer evaluates feasible recipes.
5. Owned materials are valued economically rather than at zero.
6. UI compares buy-all, use-owned, and mixed strategies when data permits.
7. User sees the exact intermediate path used by the calculation.

Journey C - Session plan

1. User chooses available capital, preferred strategy, desired risk, and an effort category.
2. Engine filters opportunities that do not fit.
3. Planner builds a ranked set without claiming exact execution time.
4. User can save selected opportunities to local history.

6. Definitions

Profit

Profit is the modeled net economic gain for a specific scenario after explicitly modeled transaction fees and acquisition costs.

Realized profit

Once history exists, realized profit is based only on recorded actual acquisition/sale values and actual applicable fees, not current market value.

Unrealized P/L

A separate statistic comparing current modeled value to recorded cost basis. It MUST NOT be mixed with realized profit.

Liquidity

A proxy derived from current order-book depth and price impact. It is not a guaranteed execution probability.

Confidence

A qualitative or component-based indication of evidence quality. It MUST NOT be represented as a mathematically precise probability unless a validated statistical model exists.

7. Non-functional requirements

- Local application starts without an external server.
- Default bind address MUST be loopback-only.
- No API key is sent to browser JavaScript.
- No API key is written to logs.

- Core calculations are deterministic for identical inputs and configuration.
- Tests MUST run without live GW2 API access.
- UI MUST support current major desktop browsers. Exact browser matrix is a release configuration item.
- Accessibility: keyboard navigation, readable contrast, semantic controls, sensible focus management.
- No decorative reuse of GW2 proprietary UI assets.

8. API verification policy

Exact endpoint schemas, scopes, current quotas, cache guidance, and field meanings are external contracts and MUST be revalidated before release.

Initial authoritative/community references to verify:

- <https://wiki.guildwars2.com/wiki/API:2/commerce/prices>
- <https://wiki.guildwars2.com/wiki/API:2/commerce/listings>
- <https://wiki.guildwars2.com/wiki/API:2/commerce/transactions>
- <https://wiki.guildwars2.com/wiki/API:2/recipes>
- <https://wiki.guildwars2.com/wiki/API:2/recipes/search>
- <https://wiki.guildwars2.com/wiki/API:2/account/bank>
- <https://wiki.guildwars2.com/wiki/API:2/account/materials>
- <https://wiki.guildwars2.com/wiki/API:2/characters/:id/crafting>
- <https://wiki.guildwars2.com/wiki/API:2/account/recipes>
- <https://wiki.guildwars2.com/wiki/API:2/tokeninfo>
- https://wiki.guildwars2.com/wiki/API:Best_practices
- https://wiki.guildwars2.com/wiki/API:Terms_of_Use

If an exact field, quota, fee behavior, or permission is uncertain, the implementation MUST mark it VERIFY and stop short of treating the assumption as authoritative.

9. Market analysis requirements

Top-of-book

The engine MAY use current aggregate prices for cheap broad candidate screening.

Order-book analysis

Detailed listings SHOULD be used for candidates that survive screening. The engine SHOULD simulate quantity-aware acquisition and liquidation across multiple price levels.

Fee configuration

Fee rules MUST be centralized in a configurable policy. The application MUST NOT scatter magic constants through analytical code.

Data freshness

Thresholds **MUST** be configurable. Suggested initial states are Fresh, Aging, Stale, Incomplete. These labels are product policy, not API facts.

10. Crafting requirements

The analyzer **MUST** respect available recipes, disciplines/ratings, recipe availability, and ingredient quantities when those facts are accessible.

For a multi-step path the engine **MUST**:

- detect cycles;
- impose a configurable maximum depth;
- memoize repeated subproblems;
- stop or truncate gracefully when the search space becomes too large;
- expose when analysis was truncated;
- compare purchased ingredient economics against owned-item opportunity cost.

11. Session planning requirements

The initial planner uses categories rather than exact execution promises:

- Very low effort
- Low effort
- Medium effort
- High effort
- Ongoing/patient

The user **MAY** override category preferences. Precise time estimates require later data collection and calibration.

12. History requirements

Local persistence **MAY** include:

- user preference profile;
- saved opportunities;
- planned operations;
- recorded actual outcomes;
- locally captured market snapshots;
- calculation versions/strategy configuration identifiers;
- account snapshots, with data minimization.

The UI **MUST** let the user clear local account-related data.

13. Future historical investment feature

The application MAY collect sampled market snapshots locally. Historical analysis should not begin as a predictive feature.

First provide descriptive measures such as observed percentile, volatility, spread persistence, drawdown, and liquidity stability. Only later consider predictive strategies, with explicit validation against out-of-sample data.

14. Future LLM boundary

No LLM integration is required for the current project. If later introduced, it MUST be an adapter around deterministic application services and MUST NOT have direct access to the GW2 API key or authority to perform mutations.

15. Completion standard

A release is not complete because the UI looks correct. A release is complete only when:

- requirements are reflected in tests;
- tests pass from a clean checkout;
- API assumptions are verified/documented;
- no secret is committed;
- rate-limit/caching behavior is observable;
- calculations are reproducible;
- read-only constraints are demonstrably enforced by architecture.

Architecture

Target stack

Recommended baseline:

- .NET 10 LTS backend with ASP.NET Core
- React + TypeScript frontend
- SQLite via EF Core or a narrowly-scoped persistence abstraction
- xUnit for .NET tests
- Playwright for browser smoke/end-to-end tests
- built-in structured logging with redaction rules

Alternative: Blazor Web App if a single-language C# stack is strongly preferred. Do not switch to Blazor solely because it looks simpler on day one; the current specification benefits from a clear browser/client boundary and TypeScript ecosystem.

Runtime topology

Browser -> ASP.NET Core local server -> application services -> deterministic analytics / SQLite / GW2 API gateway

No browser component calls GW2 authenticated endpoints directly.

Layers

Domain

Pure models and invariants. No HTTP, database, framework, or UI dependencies.

Application

Use cases and orchestration. Defines interfaces such as market data provider, account data provider, snapshot repository, clock, and secret store.

Infrastructure

GW2 HTTP implementation, caching, persistence, secret handling, filesystem configuration, external adapters.

Analytics

Pure deterministic calculators: fees, market scenarios, order-book simulation, crafting path search, opportunity score, stale-data handling.

Web

HTTP endpoints, DTO mapping, UI, validation, presentation.

Recommended repository structure

```
/
src/
  Gw2Tp.Web/
  Gw2Tp.Application/
  Gw2Tp.Domain/
  Gw2Tp.Analytics/
  Gw2Tp.Infrastructure/
tests/
  Gw2Tp.Domain.Tests/
  Gw2Tp.Application.Tests/
  Gw2Tp.Analytics.Tests/
  Gw2Tp.Infrastructure.Tests/
  Gw2Tp.IntegrationTests/
```



```
Gw2Tp.Web.E2E/  
docs/  
config/
```

External API gateway

All GW2 API calls MUST pass through one abstraction:

IGw2ApiClient

with internal components:

- request scheduler;
- cache;
- deduplicator;
- batching coordinator;
- retry/backoff policy;
- response validation;
- metrics/logging.

Feature code must never construct an ArenaNet URL itself.

API caching

Reference data: long TTL and explicit refresh/invalidation.

Market data: short TTL and freshness metadata.

Account snapshots: user-controlled refresh or moderate TTL.

All cached authenticated responses are scoped to a local account identity/key identifier and are never mixed across account contexts.

Rate-limit strategy

The request scheduler should support a configurable token bucket. Current documented/community limits must be verified before release, not hard-coded as immutable facts.

On 429:

1. record the event;
2. respect server-provided retry information when available;
3. apply bounded exponential backoff;
4. suppress duplicate concurrent requests;
5. surface a useful UI state.

Persistence

SQLite is the default local store. Use migrations and schema versioning.

Persist only information needed for the feature. Keep raw API snapshots only where they provide a clear debugging/historical value.

Money

Represent money in integer copper using a dedicated value type or equivalent. No floating-point arithmetic for gold/silver/copper calculations.

Secrets

Use an OS-backed local secret store through an abstraction. For development, support an environment-variable override that is explicitly documented as local development only.

Local network binding

Default server address: 127.0.0.1.

Do not expose the application to the LAN by default.

Error taxonomy

External failures should map to stable application error categories:

- AuthenticationFailed
- PermissionDenied
- RateLimited
- TemporarilyUnavailable
- NotFound
- InvalidRemoteData
- UnsupportedSchema
- LocalConfigurationError

Observability

Track:

- API request count by endpoint category;
- cache hit/miss;
- request latency;
- 429 count;
- failed parsing;
- market data age;
- analytics duration;
- number of candidates analyzed;
- search truncation events.

Never log API keys, authorization headers, or complete sensitive account payloads.

LLM separation

No application LLM in current scope. The coding model is an external development dependency. If a future LLM feature is added, it must consume structured application output through an adapter and must not access secrets or mutate state.

Security and Data Protection Baseline

Scope

The initial application is local and personal. This is intentionally not a multi-user web service.

For a genuinely personal/household activity, GDPR Article 2(2)(c) can exclude the activity from GDPR scope. That conclusion is context-dependent and must be reassessed if the application is commercialized, shared, hosted, or used for other persons' data.

References: - <https://eur-lex.europa.eu/eli/reg/2016/679/oj> - <https://www.cnil.fr/fr/securite-api-interfaces-de-programmation-applicative> - <https://www.cnil.fr/fr/passer-laction/garantir-la-securite-des-donnees> - <https://www.ssi.gouv.fr/entreprise/bonnes-pratiques/>

Mandatory/conditional requirements

1. Comply with current Guild Wars 2 API and website/content terms.
2. If the scope becomes non-personal, apply GDPR/French data-protection obligations appropriate to that deployment.
3. Protect API credentials against unauthorized access.
4. Apply security controls proportionate to risk.
5. Reassess legal/security scope before public or hosted deployment.

Best practices required by this project

- API key stored outside source code;
- no secret in browser JavaScript;
- loopback-only server by default;
- redacted logs;
- minimal local data retention;
- explicit clear-data action;
- dependency updates;
- test fixtures free of real API keys;
- no analytics/telemetry sent remotely without explicit future decision;

- secure default headers where compatible with local development;
- input validation on all locally exposed HTTP endpoints;
- no unsafe HTML rendering of API-controlled names or text.

API token specifics

The current GW2 tokeninfo documentation warns that the key name is not escaped and can contain HTML/JavaScript. Render it as text only.

Permissions required by the application must be declared per feature. The key validator should show missing permissions and disable corresponding functionality instead of repeatedly attempting unauthorized calls.

Data minimization

Prefer storing:

- opaque account identifier or local profile ID;
- selected account-derived facts needed for current analysis;
- timestamps;
- user-entered preferences;
- saved operation outcomes.

Avoid storing full raw authenticated payloads indefinitely.

Threat model

Threats include:

- accidental Git secret commit;
- malicious local webpage attempting to call the local server;
- XSS from API-derived strings;
- compromised local process reading secrets;
- dependency vulnerability;
- unauthorized LAN access if binding changes;
- stale/corrupt market data causing bad decisions.

Read-only enforcement

The application should not expose an HTTP client abstraction that can issue arbitrary authenticated methods/URLs. Restrict the API adapter to a typed allow-list of known GET resources.

The project should include a regression test that no write-capable API method exists in the GW2 adapter surface.

Testing Strategy

Quality layers

Unit tests

Required for all deterministic business logic:

- Money and currency conversion
- Fee policy
- Profit calculations
- ROI and capital efficiency
- Order-book depth simulation
- Liquidity metrics
- Freshness classification
- Opportunity scoring
- Session planning
- Owned-item opportunity cost
- Crafting graph/search
- Data validation and normalization

Integration tests

Use recorded fixtures or mocks for GW2 responses. Normal automated tests must not call live endpoints.

Cover:

- prices
- listings
- recipes
- account tokeninfo
- bank/materials/inventory
- character crafting/recipe availability
- transaction history where used
- 401/403/404/429/5xx
- malformed JSON
- missing/new fields

Contract/smoke tests

A controlled, manually triggered or release-stage suite MAY call the live API to confirm endpoint shape. This is not the normal CI suite.

Browser tests

Use Playwright to test the highest-value journeys:

1. dashboard loads with fixture data;

2. filters work;
3. opportunity detail explains assumptions;
4. API permission failure is understandable;
5. no secret appears in HTML or network responses;
6. data freshness is visible.

Test fixture policy

Fixtures must be:

- deterministic;
- synthetic or sanitized;
- small enough to inspect manually;
- versioned with the test that depends on them.

Specification-test coupling

Every ticket that changes behavior must update at least one of:

- acceptance tests;
- unit tests;
- integration fixtures/tests;
- browser tests;
- specification text.

The PR/commit should make the change relationship obvious.

Financial correctness gates

The following cannot ship with failing tests:

- copper arithmetic;
- fee calculation;
- order-book simulation;
- crafting economic cost;
- realized profit reconciliation.

LLM testing

No LLM application tests exist in current scope. Development-agent behavior is evaluated operationally through ticket acceptance and repository tests.

If a future LLM feature is added, deterministic invariants must still be tested outside the LLM.

UI/UX Specification

Visual direction

Modern analytical dashboard with restrained Guild Wars 2-inspired atmosphere without copying ArenaNet assets.

Use:

- dark/charcoal base;
- subtle metallic/parchment surfaces;
- restrained cyan/teal accents;
- clear gold/copper emphasis only for economic values;
- high information density with generous grouping;
- simple icons and CSS shapes instead of copied game assets.

Main screens

Dashboard

- opportunity ranking;
- user/session constraints;
- data freshness;
- last refresh;
- warning banner when data is stale/incomplete.

Opportunity detail

Show:

- exact scenario;
- acquisition assumption;
- exit assumption;
- fees;
- modeled profit;
- capital required;
- ROI;
- order-book impact;
- liquidity proxy;
- risk/confidence factors;
- freshness.

Crafting detail

Show the path as a tree or expandable graph:

output -> recipe -> intermediate ingredients -> source cost.

Highlight where owned materials are used and show their opportunity cost.

Account/settings

- API key connection state;
- permission status;
- data refresh controls;
- stored profile settings;
- clear local account data.

Interaction rules

- Do not hide critical assumptions behind tooltips only.
- Use explicit labels: **Modeled profit**, not **Profit**.
- Use **Data age** everywhere live market data appears.
- Never show a green “guaranteed” state.
- Use filter controls that preserve current selections when possible.
- Support keyboard navigation.

Desktop-first

Mobile support is deferred. The desktop layout should be responsive enough to avoid unusable overflow at narrower desktop window widths.

Qwen + MTPLX Development Workflow

Role of Qwen

Qwen is the development agent. It is not a product decision-maker and is not the application’s business-logic authority.

Context hierarchy

Load the minimum context needed:

1. docs/context/permanent-context.md
2. docs/context/milestone-context-Mx.md
3. exactly one ticket from docs/tickets/
4. the ticket prompt from docs/prompts/
5. only the relevant source files.

Do not paste the entire specification into every request.

Ticket loop

For every ticket:

1. Read the ticket and acceptance criteria.
2. Inspect current repository state.
3. Identify contradictions or missing dependencies.

4. State implementation plan briefly.
5. Implement only the ticket scope.
6. Add/update tests before declaring success.
7. Run formatter/analyzers/tests.
8. Inspect changed files for secret leakage and accidental scope expansion.
9. Report files changed, tests run, known limitations, and any specification issue.

When Qwen should stop

Qwen must stop and ask for a human decision when:

- a current API fact contradicts the specification;
- a ticket requires a write-capable external operation;
- an architectural decision must change;
- a secret is required but no secure mechanism is configured;
- tests and requirements appear mutually inconsistent;
- a dependency requires accepting an unacceptable license or security risk.

Prompt style

Prompts should be imperative, scoped, test-oriented, and explicit about non-goals. Avoid “build the app” requests.

Commit guidance

One logical ticket per commit is preferred. Suggested message:

`TKT-M3-02: add fee-aware flip calculator`

ADRs

An ADR is an Architecture Decision Record: a short document that records an important architectural decision, its context, the chosen option, alternatives, and consequences.

Create/update an ADR when a decision is durable and cross-cutting.

Review rules

Human review is required for:

- changes to API permissions;
- secrets/authentication;
- API request policy;
- financial formulas;
- persistence schema decisions;
- security behavior;

- additions to project scope.

Roadmap

Phase	Outcome	Gate
M0	External contracts and development runtime verified	No unknown critical dependency
M1	Working local skeleton	Clean build + tests
M2	Safe GW2 data gateway	Cached, throttled, fixture-tested data
M3	Deterministic market engine	Correct cop-per/fees/depth/scoring
M4	Useful desktop MVP	User can find and inspect opportunities
M5	Account/crafting mode	Feasible and economic crafting analysis
M6	Personal history	Realized profit is reproducible
M7	Historical research	Local evidence without prediction claims
M8	Release-ready local tool	Security, UX, performance, reproducible setup

Do not skip M0. It contains the model/runtime and API validation gates.

External Verification Notes

Verification date: 2026-08-19.

MTPLX

- MTPLX is Apple Silicon/macOS focused and currently documents a local OpenAI-compatible server on `127.0.0.1:8000`.
- Current MTPLX documentation says it runs MLX-native models and treats GGUF as the llama.cpp format; raw GGUF repositories are not assumed to load directly.
- Current MTPLX documentation lists Qwen 3.8 27B compatible catalog artifacts and memory guidance. Exact local model choice must be tested on the user's Mac.
- Source: <https://github.com/youssofal/MTPLX>
- Source: <https://github.com/youssofal/MTPLX/blob/main/CHANGELOG.md>

Qwen3.8-27B GGUF

- Unsloth's current model page documents Qwen3.8-27B GGUF variants and quantization sizes.
- It is a GGUF distribution, which is separate from MTPLX's native MLX runtime path.
- Source: <https://huggingface.co/unsloth/Qwen3.8-27B-GGUF>

Visual Studio on Mac

- Visual Studio for Mac was retired on 2024-08-31. Microsoft recommends Visual Studio Code on Mac.
- Source: <https://learn.microsoft.com/en-us/lifecycle/announcements/visual-studio-mac-end-of-servicing>
- Source: <https://code.visualstudio.com/docs/setup/mac>

GW2 API

The current project uses these references as the starting point for verification:

- <https://wiki.guildwars2.com/wiki/API:2/commerce/prices> - <https://wiki.guildwars2.com/wiki/API:2/commerce/listings> - <https://wiki.guildwars2.com/wiki/API:2/tokeninfo>
- https://wiki.guildwars2.com/wiki/API_key - https://wiki.guildwars2.com/wiki/API:Best_practices
- https://wiki.guildwars2.com/wiki/API:Terms_of_Use

The exact API contract must always be rechecked before release.

Architecture Decision Records

ADR-001 - Application Stack

Status

Accepted

Decision

Use ASP.NET Core/.NET 10 for the backend, React + TypeScript for the frontend, SQLite for local persistence, xUnit for .NET tests, and Playwright for browser-level tests.

Rationale

Clear separation of deterministic business logic from browser UI; strong .NET tooling; mature SQLite support; good fit for local desktop use and future hosted deployment.

Alternative

Blazor Web App remains a valid alternative if keeping one language becomes more important than the current frontend separation.

ADR-002 - Local-First Deployment

Status

Accepted

Decision

V1 runs entirely on the user's Mac with a loopback server and SQLite. No cloud server is required.

Consequences

Simpler security and privacy model; backups are the user's responsibility; future multi-user deployment will require a separate architecture review.

ADR-003 - No Application LLM in Current Scope

Status

Accepted

Decision

The application contains no LLM integration in the initial architecture. Qwen3.8-27B via MTPLX is a development agent only.

Rationale

Financial calculations and API semantics must remain deterministic, testable, reproducible, and independent of model behavior.

Future

A future LLM may explain structured results through an adapter without accessing secrets or executing actions.

ADR-004 - Single GW2 API Gateway

Status

Accepted

Decision

All Guild Wars 2 API requests go through one typed gateway abstraction with caching, batching, deduplication, scheduling, retries, and response validation.

Rationale

Prevents request multiplication, centralizes rate-limit policy, and isolates external schema changes.

ADR-005 - Integer Copper Money

Status

Accepted

Decision

Represent money as integer copper.

Rationale

Avoid floating-point rounding errors and make financial tests exact.

ADR-006 - Local Secret Storage

Status

Accepted

Decision

API credentials are stored outside source code using an OS-backed secret mechanism.

Development fallback

An environment variable may be used for local development and test execution only; it must never be committed.

ADR-007 - Read-Only External Boundary

Status

Accepted

Decision

The GW2 adapter exposes only explicitly supported GET resources. There is no generic authenticated request method available to feature code.

Rationale

Makes the project's read-only promise structural and testable.

Milestones

M0 - Discovery and external-contract validation

Goal

Validate the model/runtime path, GW2 API capabilities, permissions, fees, quotas, and project constraints before implementation.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M1 - Repository and development foundation

Goal

Create the solution, modules, tests, CI-friendly commands, local configuration, and secure local runtime skeleton.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M2 - GW2 data gateway and caching

Goal

Build the typed read-only API client, request scheduling, caching, batching, fixtures, and observability.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M3 - Deterministic market engine

Goal

Implement money, fee policy, top-of-book screening, order-book simulation, liquidity, freshness, and opportunity scoring.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M4 - Dashboard and session planning

Goal

Deliver the first genuinely useful desktop UI and session planner around market opportunities.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M5 - Account-aware analysis and crafting

Goal

Add API-key validation, account snapshots, owned-item economics, crafting feasibility, and bounded multi-step crafting analysis.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M6 - Personal history and reconciliation

Goal

Persist saved operations and realized outcomes; distinguish realized and unrealized profit; support local clearing.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M7 - Historical market data and investment research

Goal

Collect local market snapshots efficiently and provide descriptive long-term analysis without pretending to forecast the future.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

M8 - Hardening, accessibility, release readiness

Goal

Security review, browser compatibility, performance checks, documentation, backups, and a clean local release workflow.

Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

AI Context Files

Permanent Context for Qwen

Identity

You are the coding agent for a local Guild Wars 2 Trading Post analysis application.

Hard constraints

- Read-only application.
- No gameplay automation.
- No Trading Post automation.
- No API mutation/write operations.
- No secret in source code, browser code, logs, fixtures, prompts, or tests.
- Core financial truth must be deterministic.
- All GW2 API access goes through one typed gateway with caching/rate limiting.
- Money uses integer copper.
- Tests are required for business logic changes.
- Never invent undocumented GW2 API fields or quotas.
- Preserve modular boundaries.
- Do not add an application LLM; Qwen is only the development agent.

Target stack

- ASP.NET Core / .NET 10 LTS
- React + TypeScript
- SQLite
- xUnit
- Playwright for browser tests

Local development

macOS Apple Silicon. Visual Studio for Mac is retired; use Visual Studio Code or equivalent. MTPLX serves the local coding model on loopback.

Current architecture

Browser -> Web API -> Application services -> Analytics/Infrastructure -> SQLite/GW2 API.

Model/runtime note

MTPLX is MLX-native. The raw `unsloth/Qwen3.8-27B-GGUF` model is not assumed to load directly. Compatibility must be validated in M0. The application itself must not depend on MTPLX.

Required behavior

When uncertain, mark **VERIFY** and avoid inventing facts.

Prefer small, reversible changes.

Context - M0

Milestone

M0 - Discovery and external-contract validation

Goal

Validate the model/runtime path, GW2 API capabilities, permissions, fees, quotas, and project constraints before implementation.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M1

Milestone

M1 - Repository and development foundation

Goal

Create the solution, modules, tests, CI-friendly commands, local configuration, and secure local runtime skeleton.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M2

Milestone

M2 - GW2 data gateway and caching

Goal

Build the typed read-only API client, request scheduling, caching, batching, fixtures, and observability.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M3

Milestone

M3 - Deterministic market engine

Goal

Implement money, fee policy, top-of-book screening, order-book simulation, liquidity, freshness, and opportunity scoring.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M4

Milestone

M4 - Dashboard and session planning

Goal

Deliver the first genuinely useful desktop UI and session planner around market opportunities.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M5

Milestone

M5 - Account-aware analysis and crafting

Goal

Add API-key validation, account snapshots, owned-item economics, crafting feasibility, and bounded multi-step crafting analysis.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M6

Milestone

M6 - Personal history and reconciliation

Goal

Persist saved operations and realized outcomes; distinguish realized and unrealized profit; support local clearing.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M7

Milestone

M7 - Historical market data and investment research

Goal

Collect local market snapshots efficiently and provide descriptive long-term analysis without pretending to forecast the future.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Context - M8

Milestone

M8 - Hardening, accessibility, release readiness

Goal

Security review, browser compatibility, performance checks, documentation, backups, and a clean local release workflow.

Permanent context

Load docs/context/permanent-context.md.

Rules for this milestone

Implement only the tickets in this milestone. If a ticket requires an architecture change, create/update an ADR before coding beyond the smallest safe change.

Implementation Tickets and Qwen Prompts

TKT-M0-01 - Verify MTPLX/Qwen3.8-27B development model path

Milestone

M0

Goal

Determine whether the user-provided Qwen3.8-27B-GGUF can be used directly, must be converted, or should be replaced by an MTPLX-compatible Qwen3.8 27B artifact.

Dependencies

None

Acceptance criteria

- ☐ Inspect current MTPLX documentation and model compatibility behavior.
- ☐ Run the smallest safe model inspection/load command available on the Mac.
- ☐ Document whether raw GGUF loads directly.
- ☐ Document the chosen MTPLX-compatible artifact and provenance if conversion/catalog artifact is required.
- ☐ Do not modify application code to couple it to MTPLX.

Required tests

- ☐ A reproducible smoke command succeeds or fails with an explicit documented reason.
- ☐ The decision is recorded in the ticket/ADR notes.

Non-goals

- Changing the application architecture.
- Training a model.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md

- Relevant milestone context file

Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - `docs/context/permanent-context.md` - `docs/context/milestone-context-M0.md` - `docs/tickets/TKT-M0-01.md`

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Determine whether the user-provided Qwen3.8-27B-GGUF can be used directly, must be converted, or should be replaced by an MTPLX-compatible Qwen3.8 27B artifact.

Ticket-specific acceptance criteria: - Inspect current MTPLX documentation and model compatibility behavior. - Run the smallest safe model inspection/load command available on the Mac. - Document whether raw GGUF loads directly. - Document the chosen MTPLX-compatible artifact and prove-nance if conversion/catalog artifact is required. - Do not modify application code to couple it to MTPLX.

Ticket-specific non-goals: - Changing the application architecture. - Training a model.

TKT-M0-02 - Validate GW2 API endpoint and permission matrix

Milestone

M0

Goal

Create the authoritative endpoint table used by the application.

Dependencies

None

Acceptance criteria

- ☐ List every MVP endpoint, purpose, required permission, batching capability, expected freshness, and cache policy.
- ☐ Mark every uncertain fact VERIFY.
- ☐ Include prices, listings, recipes/search, account tokeninfo, bank/materials, character crafting/recipes, and any other endpoint actually required.
- ☐ No undocumented endpoint is added as a dependency.

Required tests

- ☐ Review table against current documentation URLs.
- ☐ At least one saved response fixture exists for each endpoint category later required by M2.

Non-goals

- Implementing the HTTP client.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M0.md - docs/tickets/TKT-M0-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create the authoritative endpoint table used by the application.

Ticket-specific acceptance criteria: - List every MVP endpoint, purpose, required permission, batching capability, expected freshness, and cache policy. - Mark every uncertain fact VERIFY. - Include prices, listings, recipes/search, account tokeninfo, bank/materials, character crafting/recipes, and any other endpoint actually required. - No undocumented endpoint is added as a dependency.

Ticket-specific non-goals: - Implementing the HTTP client.

TKT-M0-03 - Validate API quotas, caching guidance, and error behavior

Milestone

M0

Goal

Turn rate-limit and external-contract assumptions into measurable configuration requirements.

Dependencies

None

Acceptance criteria

- ☐ Document current rate-limit guidance from authoritative/community documentation.
- ☐ Define application-level configurable scheduler parameters without hard-coding an unverified quota.
- ☐ Document 429 handling and retry behavior.
- ☐ Define which live verification is safe and how it will be performed without deliberately stressing the API.

Required tests

- ☐ A rate-limit policy document exists.
- ☐ A mock 429 scenario is specified.

Non-goals

- Load testing the real GW2 API.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M0.md - docs/tickets/TKT-M0-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Turn rate-limit and external-contract assumptions into measurable configuration requirements.

Ticket-specific acceptance criteria: - Document current rate-limit guidance from authoritative/community documentation. - Define application-level configurable scheduler parameters without hard-coding an unverified quota. - Document 429 handling and retry behavior. - Define which live verification is safe and how it will be performed without deliberately stressing the API.

Ticket-specific non-goals: - Load testing the real GW2 API.

TKT-M0-04 - Validate legal/security scope and read-only boundary

Milestone

M0

Goal

Produce the release gate for terms, API-key handling, local security, and the strict read-only boundary.

Dependencies

None

Acceptance criteria

- ☐ Document current API terms references.
- ☐ Document personal-use GDPR/French security assumptions and what changes for public deployment.
- ☐ Define minimum API-key permissions by feature.
- ☐ Define explicit prohibited operations and a testable architecture rule preventing generic writes.

Required tests

- ☐ Security checklist exists.
- ☐ Read-only regression test idea is recorded.

Non-goals

- Legal advice beyond documenting requirements and assumptions.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M0.md - docs/tickets/TKT-M0-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Produce the release gate for terms, API-key handling, local security, and the strict read-only boundary.

Ticket-specific acceptance criteria: - Document current API terms references. - Document personal-use GDPR/French security assumptions and what changes for public deployment. - Define minimum API-key permissions by feature. - Define explicit prohibited operations and a testable architecture rule preventing generic writes.

Ticket-specific non-goals: - Legal advice beyond documenting requirements and assumptions.

TKT-M1-01 - Initialize repository and solution skeleton

Milestone

M1

Goal

Create the .NET/React repository structure and baseline developer tooling.

Dependencies

M0-04

Acceptance criteria

- ☐ Create src and tests structure from architecture.md.
- ☐ Create solution/project files with nullable and warnings-as-errors enabled where feasible.
- ☐ Create a minimal React frontend and backend that can be started locally.
- ☐ Add root README development commands.
- ☐ No business logic yet.

Required tests

- ☐ Clean checkout builds backend.
- ☐ Frontend installs/builds.
- ☐ Smoke test confirms local app starts.

Non-goals

- Implementing GW2 API access.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files

changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create the .NET/React repository structure and base-line developer tooling.

Ticket-specific acceptance criteria: - Create src and tests structure from architecture.md. - Create solution/project files with nullable and warnings-as-errors enabled where feasible. - Create a minimal React frontend and backend that can be started locally. - Add root README development commands. - No business logic yet.

Ticket-specific non-goals: - Implementing GW2 API access.

TKT-M1-02 - Add local configuration and secret-store abstraction

Milestone

M1

Goal

Create configuration interfaces that never require putting the API key in source control.

Dependencies

M1-01

Acceptance criteria

- ☐ Add an abstraction for secret retrieval/storage.
- ☐ Support a local development environment variable override.
- ☐ Ensure API key values are excluded from logs and browser responses.
- ☐ Add clear configuration error messages when a secret is missing.

Required tests

- ☐ Missing-secret test.
- ☐ Log redaction test.
- ☐ Browser endpoint does not expose secret test.

Non-goals

- Adding cloud secret management.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create configuration interfaces that never require putting the API key in source control.

Ticket-specific acceptance criteria: - Add an abstraction for secret retrieval/storage. - Support a local development environment variable override. - Ensure API key values are excluded from logs and browser responses. - Add clear configuration error messages when a secret is missing.

Ticket-specific non-goals: - Adding cloud secret management.

TKT-M1-03 - Add test infrastructure and fixture conventions

Milestone

M1

Goal

Create reusable test helpers and a fixture policy.

Dependencies

M1-01

Acceptance criteria

- ☐ Create unit, integration, and browser-test projects or equivalent harnesses.
- ☐ Create fixture folders and naming conventions.
- ☐ Create deterministic clock/test data helpers where needed.
- ☐ Document test commands.

Required tests

- ☐ All test projects execute with zero tests failing.
- ☐ One representative test exists at each layer.

Non-goals

- Real API calls in normal test execution.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create reusable test helpers and a fixture policy.

Ticket-specific acceptance criteria: - Create unit, integration, and browser-test projects or equivalent harnesses. - Create fixture folders and naming conventions. - Create deterministic clock/test data helpers where needed. - Document test commands.

Ticket-specific non-goals: - Real API calls in normal test execution.

TKT-M1-04 - Add local-only runtime and base-line security middleware

Milestone

M1

Goal

Make loopback-only operation and safe HTTP defaults explicit.

Dependencies

M1-01,M1-02

Acceptance criteria

- ☐ Default development server binds to loopback.
- ☐ Add safe response headers compatible with the local app.
- ☐ Validate inbound query/body values at API boundaries.
- ☐ Document how to intentionally change the binding and why doing so is not recommended.

Required tests

- ☐ Configuration test asserts loopback default.
- ☐ Security header smoke test.

Non-goals

- LAN/public hosting support.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. -

Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Make loopback-only operation and safe HTTP defaults explicit.

Ticket-specific acceptance criteria: - Default development server binds to loopback. - Add safe response headers compatible with the local app. - Validate inbound query/body values at API boundaries. - Document how to intentionally change the binding and why doing so is not recommended.

Ticket-specific non-goals: - LAN/public hosting support.

TKT-M2-01 - Implement typed GW2 API client for public market data

Milestone

M2

Goal

Implement typed read-only requests for prices and listings based on verified schemas.

Dependencies

M0-02,M0-03,M1-03

Acceptance criteria

- ☐ Only explicit GET methods are exposed.
- ☐ External DTOs remain separate from domain models.
- ☐ Batch IDs are supported where the verified endpoint allows them.
- ☐ Unexpected fields do not crash parsing unnecessarily.
- ☐ Errors map to stable error categories.

Required tests

- ☐ Fixture tests for 200 responses.
- ☐ Malformed JSON test.
- ☐ HTTP error mapping tests.

Non-goals

- Account/authenticated endpoints other than those explicitly needed by later tickets.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files

changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Implement typed read-only requests for prices and listings based on verified schemas.

Ticket-specific acceptance criteria: - Only explicit GET methods are exposed.
- External DTOs remain separate from domain models. - Batch IDs are supported where the verified endpoint allows them. - Unexpected fields do not crash parsing unnecessarily. - Errors map to stable error categories.

Ticket-specific non-goals: - Account/authenticated endpoints other than those explicitly needed by later tickets.

TKT-M2-02 - Build request scheduler, deduplication, and retry policy

Milestone

M2

Goal

Centralize GW2 request rate management.

Dependencies

M2-01

Acceptance criteria

- ☐ Requests are scheduled through one component.
- ☐ Concurrent identical requests are deduplicated.
- ☐ 429 responses trigger bounded backoff.
- ☐ Retry rules distinguish transient from permanent failures.
- ☐ Configured quotas are not hard-coded from an unverified assumption.

Required tests

- ☐ Concurrency deduplication test.
- ☐ 429 backoff test.
- ☐ No retry for permanent auth/permission failure.

Non-goals

- Stress-testing live GW2.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Centralize GW2 request rate management.

Ticket-specific acceptance criteria: - Requests are scheduled through one component. - Concurrent identical requests are deduplicated. - 429 responses trigger bounded backoff. - Retry rules distinguish transient from permanent failures. - Configured quotas are not hard-coded from an unverified assumption.

Ticket-specific non-goals: - Stress-testing live GW2.

TKT-M2-03 - Implement cache with freshness metadata

Milestone

M2

Goal

Cache public market responses and expose data age.

Dependencies

M2-01,M2-02

Acceptance criteria

- ☐ Cache entries have capture time and expiry policy.
- ☐ Cache hit avoids network request.
- ☐ Cache invalidation/refresh is deterministic.
- ☐ Data freshness is available to analytics and UI.

Required tests

- ☐ Hit/miss tests.
- ☐ Expiry test.
- ☐ Concurrent cache fill test.

Non-goals

- Historical market database.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Cache public market responses and expose data age.

Ticket-specific acceptance criteria: - Cache entries have capture time and expiry policy. - Cache hit avoids network request. - Cache invalidation/refresh is deterministic. - Data freshness is available to analytics and UI.

Ticket-specific non-goals: - Historical market database.

TKT-M2-04 - Add market-data observability and diagnostics

Milestone

M2

Goal

Make API use and cache behavior auditable.

Dependencies

M2-02,M2-03

Acceptance criteria

- ☐ Track request counts, cache hits/misses, latency, 429s, and parsing failures.
- ☐ Never log API keys or authorization headers.
- ☐ Provide a local diagnostic view or structured diagnostic endpoint that is safe to display.

Required tests

- ☐ Sensitive-value redaction test.
- ☐ Counter increments on mock requests.

Non-goals

- Remote telemetry.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one

exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Make API use and cache behavior auditable.

Ticket-specific acceptance criteria: - Track request counts, cache hits/misses, latency, 429s, and parsing failures. - Never log API keys or authorization headers. - Provide a local diagnostic view or structured diagnostic endpoint that is safe to display.

Ticket-specific non-goals: - Remote telemetry.

TKT-M3-01 - Implement Money and fee policy

Milestone

M3

Goal

Create exact copper arithmetic and centralized transaction-fee policy.

Dependencies

M2-01

Acceptance criteria

- ☐ Money is represented in integer copper.
- ☐ No floating-point arithmetic is used for money calculations.
- ☐ Fee policy is isolated and configurable.
- ☐ Profit formulas document scenario semantics.

Required tests

- ☐ Representative fee/profit examples.
- ☐ Boundary/large-value tests.
- ☐ Rounding tests.

Non-goals

- Hard-coding unexplained fee constants throughout the application.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create exact copper arithmetic and centralized transaction-fee policy.

Ticket-specific acceptance criteria: - Money is represented in integer copper. - No floating-point arithmetic is used for money calculations. - Fee policy is isolated and configurable. - Profit formulas document scenario semantics.

Ticket-specific non-goals: - Hard-coding unexplained fee constants throughout the application.

TKT-M3-02 - Implement order-book quantity simulation

Milestone

M3

Goal

Calculate acquisition and liquidation scenarios across order-book levels.

Dependencies

M2-01,M3-01

Acceptance criteria

- ☐ Simulate quantity across multiple buy/sell levels.
- ☐ Calculate weighted average execution price and price impact.
- ☐ Handle insufficient depth explicitly.
- ☐ Return transparent assumptions and remaining quantity.

Required tests

- ☐ Single-level.
- ☐ Multiple-level.
- ☐ Partial-depth.
- ☐ Empty-book.
- ☐ Large quantity.

Non-goals

- Claiming guaranteed real-world fill behavior.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Calculate acquisition and liquidation scenarios across order-book levels.

Ticket-specific acceptance criteria: - Simulate quantity across multiple buy/sell levels. - Calculate weighted average execution price and price impact. - Handle insufficient depth explicitly. - Return transparent assumptions and remaining quantity.

Ticket-specific non-goals: - Claiming guaranteed real-world fill behavior.

TKT-M3-03 - Implement flip profitability and liquidity analysis

Milestone

M3

Goal

Generate deterministic flip scenarios and a liquidity proxy.

Dependencies

M3-01,M3-02

Acceptance criteria

- ☐ Compute modeled net profit, ROI, capital required, price impact, and liquidity metrics.
- ☐ Support configurable minimum profit and capital constraints.
- ☐ Mark missing/stale data as unusable or lower confidence according to policy.
- ☐ Each result explains the scenario used.

Required tests

- ☐ Known-good fixture.
- ☐ Known-bad negative-profit fixture.
- ☐ Stale-data fixture.
- ☐ Insufficient-depth fixture.

Non-goals

- Automated orders.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Generate deterministic flip scenarios and a liquidity proxy.

Ticket-specific acceptance criteria: - Compute modeled net profit, ROI, capital required, price impact, and liquidity metrics. - Support configurable minimum profit and capital constraints. - Mark missing/stale data as unusable or lower confidence according to policy. - Each result explains the scenario used.

Ticket-specific non-goals: - Automated orders.

TKT-M3-04 - Implement deterministic opportunity scoring

Milestone

M3

Goal

Rank opportunities using transparent weights and penalties.

Dependencies

M3-03

Acceptance criteria

- ☐ Score is deterministic for identical inputs/configuration.
- ☐ Weights can be configured without changing code.
- ☐ Score includes at least profit, capital efficiency, liquidity, freshness, risk, and complexity where applicable.
- ☐ UI-facing explanation metadata is produced.

Required tests

- ☐ Ordering tests.
- ☐ Tie tests.
- ☐ Weight sensitivity tests.
- ☐ Stale-data penalty test.

Non-goals

- LLM-generated ranking.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Rank opportunities using transparent weights and penalties.

Ticket-specific acceptance criteria: - Score is deterministic for identical inputs/configuration. - Weights can be configured without changing code. - Score includes at least profit, capital efficiency, liquidity, freshness, risk, and complexity where applicable. - UI-facing explanation metadata is produced.

Ticket-specific non-goals: - LLM-generated ranking.

TKT-M4-01 - Create market dashboard and filters

Milestone

M4

Goal

Build the main desktop dashboard around deterministic opportunities.

Dependencies

M3-04

Acceptance criteria

- ☐ Show ranked opportunities with capital, modeled profit, ROI, liquidity, risk/confidence, and data age.
- ☐ Provide filters for capital, minimum profit, strategy, risk, and freshness.
- ☐ No unsupported guarantee language.
- ☐ Empty/loading/error states are explicit.

Required tests

- ☐ Component tests for filters.

- Browser smoke test for dashboard.

Non-goals

- Mobile-first UI.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Build the main desktop dashboard around deterministic opportunities.

Ticket-specific acceptance criteria: - Show ranked opportunities with capital, modeled profit, ROI, liquidity, risk/confidence, and data age. - Provide filters for capital, minimum profit, strategy, risk, and freshness. - No unsupported guarantee language. - Empty/loading/error states are explicit.

Ticket-specific non-goals: - Mobile-first UI.

TKT-M4-02 - Create opportunity detail view

Milestone

M4

Goal

Expose calculation details and assumptions.

Dependencies

M4-01

Acceptance criteria

- ☐ Show acquisition/exit assumptions, fees, modeled profit, ROI, capital, order-book impact, liquidity, and data age.
- ☐ Provide a human-readable calculation breakdown.
- ☐ Clearly distinguish scenario values from actual outcomes.

Required tests

- ☐ Detail rendering test.
- ☐ Fixture comparison test.

Non-goals

- Executing actions from the detail page.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Expose calculation details and assumptions.

Ticket-specific acceptance criteria: - Show acquisition/exit assumptions, fees, modeled profit, ROI, capital, order-book impact, liquidity, and data age. - Provide a human-readable calculation breakdown. - Clearly distinguish scenario values from actual outcomes.

Ticket-specific non-goals: - Executing actions from the detail page.

TKT-M4-03 - Implement user session preferences

Milestone

M4

Goal

Persist user capital and opportunity preferences locally.

Dependencies

M4-01

Acceptance criteria

- ☐ Store capital, minimum profit, risk preference, strategy preference, and allocation constraints locally.
- ☐ Validate numeric ranges and sensible defaults.
- ☐ Changing preferences re-ranks or filters results deterministically.

Required tests

- ☐ Persistence test.
- ☐ Validation test.
- ☐ Preference-to-score/filter test.

Non-goals

- Remote user accounts.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. -

Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Persist user capital and opportunity preferences locally.

Ticket-specific acceptance criteria: - Store capital, minimum profit, risk preference, strategy preference, and allocation constraints locally. - Validate numeric ranges and sensible defaults. - Changing preferences re-ranks or filters results deterministically.

Ticket-specific non-goals: - Remote user accounts.

TKT-M4-04 - Implement session planner with coarse effort categories

Milestone

M4

Goal

Turn constraints into a practical shortlist without pretending to know execution time.

Dependencies

M4-03,M3-04

Acceptance criteria

- ☐ Support effort categories: very low, low, medium, high, ongoing/patient.
- ☐ Planner respects capital/risk/preferences.
- ☐ Planner produces an ordered session shortlist.
- ☐ UI states that categories are approximations, not time guarantees.

Required tests

- ☐ Constraint tests.
- ☐ Insufficient-capital test.
- ☐ Risk-filter test.

Non-goals

- Precise execution-time prediction.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files

changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Turn constraints into a practical shortlist without pretending to know execution time.

Ticket-specific acceptance criteria: - Support effort categories: very low, low, medium, high, ongoing/patient. - Planner respects capital/risk/preferences. - Planner produces an ordered session shortlist. - UI states that categories are approximations, not time guarantees.

Ticket-specific non-goals: - Precise execution-time prediction.

TKT-M5-01 - Implement token validation and permission-aware account access

Milestone

M5

Goal

Validate an API key and expose only permitted features.

Dependencies

M0-02,M0-04,M1-02,M2-02

Acceptance criteria

- ☐ Use tokeninfo or the verified equivalent to validate the key.
- ☐ Display safe permission status.
- ☐ Never return the key to the browser.
- ☐ Disable unsupported account features when scopes are missing.
- ☐ Render token metadata such as key name as text, not HTML.

Required tests

- ☐ Valid token fixture.
- ☐ Missing permission fixture.
- ☐ Malformed metadata/XSS string fixture.
- ☐ Secret non-disclosure test.

Non-goals

- Storing the API key in frontend localStorage.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Validate an API key and expose only permitted features.

Ticket-specific acceptance criteria: - Use tokeninfo or the verified equivalent to validate the key. - Display safe permission status. - Never return the key to the browser. - Disable unsupported account features when scopes are missing. - Render token metadata such as key name as text, not HTML.

Ticket-specific non-goals: - Storing the API key in frontend localStorage.

TKT-M5-02 - Implement account snapshot ingestion

Milestone

M5

Goal

Add bank/materials and relevant character data with caching and local scoping.

Dependencies

M5-01,M2-03

Acceptance criteria

- ☐ Fetch only endpoints required for enabled account features.
- ☐ Cache account snapshots separately from public market data.
- ☐ Associate snapshots with a local account profile identifier.
- ☐ Handle missing permissions and partial data gracefully.

Required tests

- ☐ Fixture ingestion tests.
- ☐ Permission failure test.
- ☐ Cache isolation test.

Non-goals

- Persisting unnecessary full raw payloads indefinitely.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Add bank/materials and relevant character data with caching and local scoping.

Ticket-specific acceptance criteria: - Fetch only endpoints required for enabled account features. - Cache account snapshots separately from public market data. - Associate snapshots with a local account profile identifier. - Handle missing permissions and partial data gracefully.

Ticket-specific non-goals: - Persisting unnecessary full raw payloads indefinitely.

TKT-M5-03 - Implement owned-item opportunity cost

Milestone

M5

Goal

Value owned materials economically and compare with buying them.

Dependencies

M3-01,M5-02

Acceptance criteria

- ☐ Owned materials are never assumed free.
- ☐ Compute a realizable economic value using configured market evidence.
- ☐ Support owned/buy/mixed strategies where data is sufficient.
- ☐ Flag bound/unavailable/non-sellable items where relevant data exists.

Required tests

- ☐ Owned vs bought example.
- ☐ Partial owned stock.
- ☐ Unavailable price.
- ☐ Non-economic ownership restriction.

Non-goals

- Assuming all inventory is sellable merely because it has an item ID.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. -

Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Value owned materials economically and compare with buying them.

Ticket-specific acceptance criteria: - Owned materials are never assumed free. - Compute a realizable economic value using configured market evidence. - Support owned/buy/mixed strategies where data is sufficient. - Flag bound/unavailable/non-sellable items where relevant data exists.

Ticket-specific non-goals: - Assuming all inventory is sellable merely because it has an item ID.

TKT-M5-04 - Implement bounded crafting path search

Milestone

M5

Goal

Analyze profitable crafting paths with feasibility constraints and controlled recursion.

Dependencies

M5-02,M5-03,M3-03

Acceptance criteria

- ☐ Recipe graph represents output/ingredient relationships.
- ☐ Apply discipline/rating and recipe availability constraints when verified data exists.

- ☐ Search detects cycles and has configurable depth/candidate limits.
- ☐ Memoization prevents repeated subproblem explosion.
- ☐ Analysis reports truncation/unknowns explicitly.
- ☐ Compare purchase cost versus owned-material opportunity cost.

Required tests

- ☐ Simple recipe.
- ☐ Multi-step recipe.
- ☐ Cycle.
- ☐ Depth cap.
- ☐ Alternative path.
- ☐ Mixed owned/purchased ingredients.

Non-goals

- Exhaustive unlimited world optimization.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Analyze profitable crafting paths with feasibility constraints and controlled recursion.

Ticket-specific acceptance criteria: - Recipe graph represents output/ingredient relationships. - Apply discipline/rating and recipe availability constraints when verified data exists. - Search detects cycles and has configurable depth/candidate limits. - Memoization prevents repeated subproblem explosion. - Analysis reports truncation/unknowns explicitly. - Compare purchase cost versus owned-material opportunity cost.

Ticket-specific non-goals: - Exhaustive unlimited world optimization.

TKT-M6-01 - Create operation history model and local persistence

Milestone

M6

Goal

Store saved/planned operations and their calculation context.

Dependencies

M4-04

Acceptance criteria

- ☐ Store operations with timestamps and calculation/configuration version identifiers.
- ☐ Preserve the scenario assumptions used at save time.
- ☐ Allow user to mark status such as planned/in-progress/completed/cancelled.

Required tests

- ☐ Persistence round trip.
- ☐ Schema migration test.

Non-goals

- Syncing history to a cloud service.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Store saved/planned operations and their calculation context.

Ticket-specific acceptance criteria: - Store operations with timestamps and calculation/configuration version identifiers. - Preserve the scenario assumptions used at save time. - Allow user to mark status such as planned/in-progress/completed/cancelled.

Ticket-specific non-goals: - Syncing history to a cloud service.

TKT-M6-02 - Implement realized profit reconciliation

Milestone

M6

Goal

Calculate realized profit separately from unrealized P/L.

Dependencies

M6-01,M3-01

Acceptance criteria

- ☐ Realized profit uses recorded actual acquisition/sale values and fees.
- ☐ Unrealized value is separately labeled.
- ☐ Incomplete operations are not silently counted as realized profit.
- ☐ Profit statistics are reproducible from stored records.

Required tests

- ☐ Complete trade.
- ☐ Partial trade.
- ☐ Cancelled trade.
- ☐ Fee variation.
- ☐ Unrealized-only record.

Non-goals

- Claiming lifetime profit before local history starts.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Calculate realized profit separately from unrealized P/L.

Ticket-specific acceptance criteria: - Realized profit uses recorded actual acquisition/sale values and fees. - Unrealized value is separately labeled. - Incomplete operations are not silently counted as realized profit. - Profit statistics are reproducible from stored records.

Ticket-specific non-goals: - Claiming lifetime profit before local history starts.

TKT-M6-03 - Build personal dashboard/history statistics

Milestone

M6

Goal

Show useful local statistics without overstating their meaning.

Dependencies

M6-02

Acceptance criteria

- ☐ Show operation count, realized profit since first recorded use, average modeled/realized metrics where justified, and completion rate where defined.
- ☐ Clearly show the historical coverage period.
- ☐ Provide empty-state messaging for insufficient history.

Required tests

- ☐ Statistics fixtures.
- ☐ Empty-history test.

Non-goals

- Backfilling unknown lifetime history.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Show useful local statistics without overstating their meaning.

Ticket-specific acceptance criteria: - Show operation count, realized profit since first recorded use, average modeled/realized metrics where justified, and completion rate where defined. - Clearly show the historical coverage period. - Provide empty-state messaging for insufficient history.

Ticket-specific non-goals: - Backfilling unknown lifetime history.

TKT-M6-04 - Implement local data clearing and backup guidance

Milestone

M6

Goal

Give the user control over local account/history data.

Dependencies

M6-01,M5-02

Acceptance criteria

- ☐ Provide a user-visible clear account data action.
- ☐ Distinguish account snapshots from public cache/history where sensible.
- ☐ Document SQLite backup/restore at a user level.
- ☐ No data is silently uploaded.

Required tests

- ☐ Clear-data integration test.
- ☐ Backup documentation review.

Non-goals

- Automated cloud backup.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. -

Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Give the user control over local account/history data.

Ticket-specific acceptance criteria: - Provide a user-visible clear account data action. - Distinguish account snapshots from public cache/history where sensible. - Document SQLite backup/restore at a user level. - No data is silently uploaded.

Ticket-specific non-goals: - Automated cloud backup.

TKT-M7-01 - Design local market snapshot schema and sampling policy

Milestone

M7

Goal

Create the foundation for historical data without over-collecting.

Dependencies

M2-03,M6-04

Acceptance criteria

- ☐ Store timestamped price/order-book snapshots with source freshness.
- ☐ Define sampling policy by item class/watchlist.
- ☐ Estimate local storage growth.
- ☐ Allow future changes without corrupting existing history.

Required tests

- ☐ Schema migration test.
- ☐ Sampling policy unit tests.

Non-goals

- Full-frequency capture of every item.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M7.md - docs/tickets/TKT-M7-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create the foundation for historical data without over-collecting.

Ticket-specific acceptance criteria: - Store timestamped price/order-book snapshots with source freshness. - Define sampling policy by item class/watchlist. - Estimate local storage growth. - Allow future changes without corrupting existing history.

Ticket-specific non-goals: - Full-frequency capture of every item.

TKT-M7-02 - Implement economical historical collection scheduler

Milestone

M7

Goal

Collect snapshots while minimizing API requests.

Dependencies

M7-01,M2-02

Acceptance criteria

- ☐ High-interest/watchlist items can be sampled more frequently.
- ☐ Low-interest items use a lower sampling rate.
- ☐ Scheduler respects API request budget.
- ☐ Collection pauses cleanly on rate limiting or application shutdown.

Required tests

- ☐ Schedule calculation tests.
- ☐ Request budget tests.
- ☐ Pause/resume tests.

Non-goals

- Aggressive historical scraping.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md

- Relevant milestone context file

Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - `docs/context/permanent-context.md` - `docs/context/milestone-context-M7.md` - `docs/tickets/TKT-M7-02.md`

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Collect snapshots while minimizing API requests.

Ticket-specific acceptance criteria: - High-interest/watchlist items can be sampled more frequently. - Low-interest items use a lower sampling rate. - Scheduler respects API request budget. - Collection pauses cleanly on rate limiting or application shutdown.

Ticket-specific non-goals: - Aggressive historical scraping.

TKT-M7-03 - Implement descriptive historical analytics

Milestone

M7

Goal

Compute percentiles, volatility, drawdown, spread persistence, and liquidity stability from local observations.

Dependencies

M7-01,M7-02

Acceptance criteria

- ☐ Historical metrics are computed only from available local observations.
- ☐ Sufficient sample checks prevent misleading statistics.
- ☐ Charts/metrics disclose observation window and sample count.
- ☐ No claim of future prediction is made.

Required tests

- ☐ Known time series metrics.
- ☐ Insufficient sample test.
- ☐ Missing observation test.

Non-goals

- Machine-learning price forecasts.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M7.md - docs/tickets/TKT-M7-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Compute percentiles, volatility, drawdown, spread persistence, and liquidity stability from local observations.

Ticket-specific acceptance criteria: - Historical metrics are computed only from available local observations. - Sufficient sample checks prevent misleading statistics. - Charts/metrics disclose observation window and sample count. - No claim of future prediction is made.

Ticket-specific non-goals: - Machine-learning price forecasts.

TKT-M7-04 - Create investment-research view with uncertainty

Milestone

M7

Goal

Present long-term observations as evidence, not certainty.

Dependencies

M7-03,M4-02

Acceptance criteria

- ☐ Show historical range/percentile and liquidity context.
- ☐ Show data coverage and sample count.
- ☐ Use language such as observed/estimated rather than guaranteed.
- ☐ Allow the user to compare a small watchlist.

Required tests

- ☐ Coverage disclosure test.
- ☐ Watchlist comparison test.

Non-goals

- Automated investment advice presented as fact.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M7.md - docs/tickets/TKT-M7-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code,

browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Present long-term observations as evidence, not certainty.

Ticket-specific acceptance criteria: - Show historical range/percentile and liquidity context. - Show data coverage and sample count. - Use language such as observed/estimated rather than guaranteed. - Allow the user to compare a small watchlist.

Ticket-specific non-goals: - Automated investment advice presented as fact.

TKT-M8-01 - Perform security review and secret-leak audit

Milestone

M8

Goal

Verify no API key or sensitive account data leaks through code, logs, browser, fixtures, or Git.

Dependencies

M6-04,M5-01

Acceptance criteria

- ☐ Run secret scanning on repository.
- ☐ Inspect network/browser responses for API-key disclosure.

- ☐ Inspect logs and exception pages for secrets.
- ☐ Confirm local bind/security defaults.
- ☐ Document remaining risks.

Required tests

- ☐ Automated secret-pattern test where appropriate.
- ☐ Manual security checklist sign-off.

Non-goals

- Guaranteeing zero vulnerabilities.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration,

or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Verify no API key or sensitive account data leaks through code, logs, browser, fixtures, or Git.

Ticket-specific acceptance criteria: - Run secret scanning on repository. - Inspect network/browser responses for API-key disclosure. - Inspect logs and exception pages for secrets. - Confirm local bind/security defaults. - Document remaining risks.

Ticket-specific non-goals: - Guaranteeing zero vulnerabilities.

TKT-M8-02 - Perform browser/accessibility and UX hardening

Milestone

M8

Goal

Validate the desktop UI across target browsers and accessibility basics.

Dependencies

M4-04,M5-04,M6-03

Acceptance criteria

- ☐ Keyboard navigation works for primary flows.
- ☐ Focus states are visible.
- ☐ Critical information meets the chosen contrast standard.
- ☐ Loading/error/empty states are understandable.
- ☐ Run smoke tests against the documented browser matrix.

Required tests

- ☐ Playwright smoke suite.
- ☐ Accessibility scan appropriate to project toolchain.

Non-goals

- Mobile-first redesign.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Validate the desktop UI across target browsers and accessibility basics.

Ticket-specific acceptance criteria: - Keyboard navigation works for primary flows. - Focus states are visible. - Critical information meets the chosen contrast standard. - Loading/error/empty states are understandable. - Run smoke tests against the documented browser matrix.

Ticket-specific non-goals: - Mobile-first redesign.

TKT-M8-03 - Performance and API-request efficiency review

Milestone

M8

Goal

Ensure the MVP remains responsive and API-efficient.

Dependencies

M3-04,M5-04,M7-02

Acceptance criteria

- ☐ Benchmark representative analytical workloads with local fixtures.
- ☐ Confirm candidate screening prevents unnecessary deep listings requests.
- ☐ Confirm cache and request deduplication behavior.
- ☐ Document any known computational limits for crafting search.

Required tests

- ☐ Performance smoke benchmark.
- ☐ Request-count regression test.

Non-goals

- Optimizing prematurely at the cost of correctness.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Ensure the MVP remains responsive and API-efficient.

Ticket-specific acceptance criteria: - Benchmark representative analytical workloads with local fixtures. - Confirm candidate screening prevents unnecessary deep listings requests. - Confirm cache and request deduplication behavior. - Document any known computational limits for crafting search.

Ticket-specific non-goals: - Optimizing prematurely at the cost of correctness.

TKT-M8-04 - Prepare local release package and handoff documentation

Milestone

M8

Goal

Make the project reproducible for the owner.

Dependencies

M8-01,M8-02,M8-03

Acceptance criteria

- ☐ One documented setup path works on a clean Mac environment.
- ☐ One documented run/test command set exists.
- ☐ Configuration and backup instructions are complete.
- ☐ Known limitations and deferred features are listed.
- ☐ No secrets or private data are included in the release package.

Required tests

- ☐ Clean-environment checklist.
- ☐ Release smoke test.

Non-goals

- Public deployment.

Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

Dedicated execution prompt

You are the implementation agent for the GW2 Trading Post Analyst project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add game-play or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. -

Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Make the project reproducible for the owner.

Ticket-specific acceptance criteria: - One documented setup path works on a clean Mac environment. - One documented run/test command set exists. - Configuration and backup instructions are complete. - Known limitations and deferred features are listed. - No secrets or private data are included in the release package.

Ticket-specific non-goals: - Public deployment.